⚓ AIS Ship Tracker

Real-time ship tracking

Kafka · PostgreSQL · Python
Docker · GitHub Actions · AWS EC2 · AWS CloudWatch
Flask · Streamlit

Capstone Project · Data Engineering Bootcamp · Thomas Junge · 2026

Project Overview

- Subscribe to 35 000+ live ship positions via the aisstream.io WebSocket API
- Consume all AIS message types through Apache Kafka
- Clean & transform data, persist live positions and static ship data in PostgreSQL
- Persist position history for individual ships (by configuration table) in PostgreSQL
- Flask webapp to show ships/routes on map, Streamlit analytics dashboard for some statistics
- Fully containerised with Docker Compose · deployed to AWS EC2 · logging in AWS CloudWatch
- Automated CI/CD pipeline via GitHub Actions

Data Source – AIS Protocol

What is AIS?

Automatic Identification System, mandated on all large vessels globally

How it works

Ships broadcast VHF radio signals captured by satellites and shore stations

27 message types

PositionReport · ShipStaticData · SafetyMessage · StandardClassBPosReport …

Our source

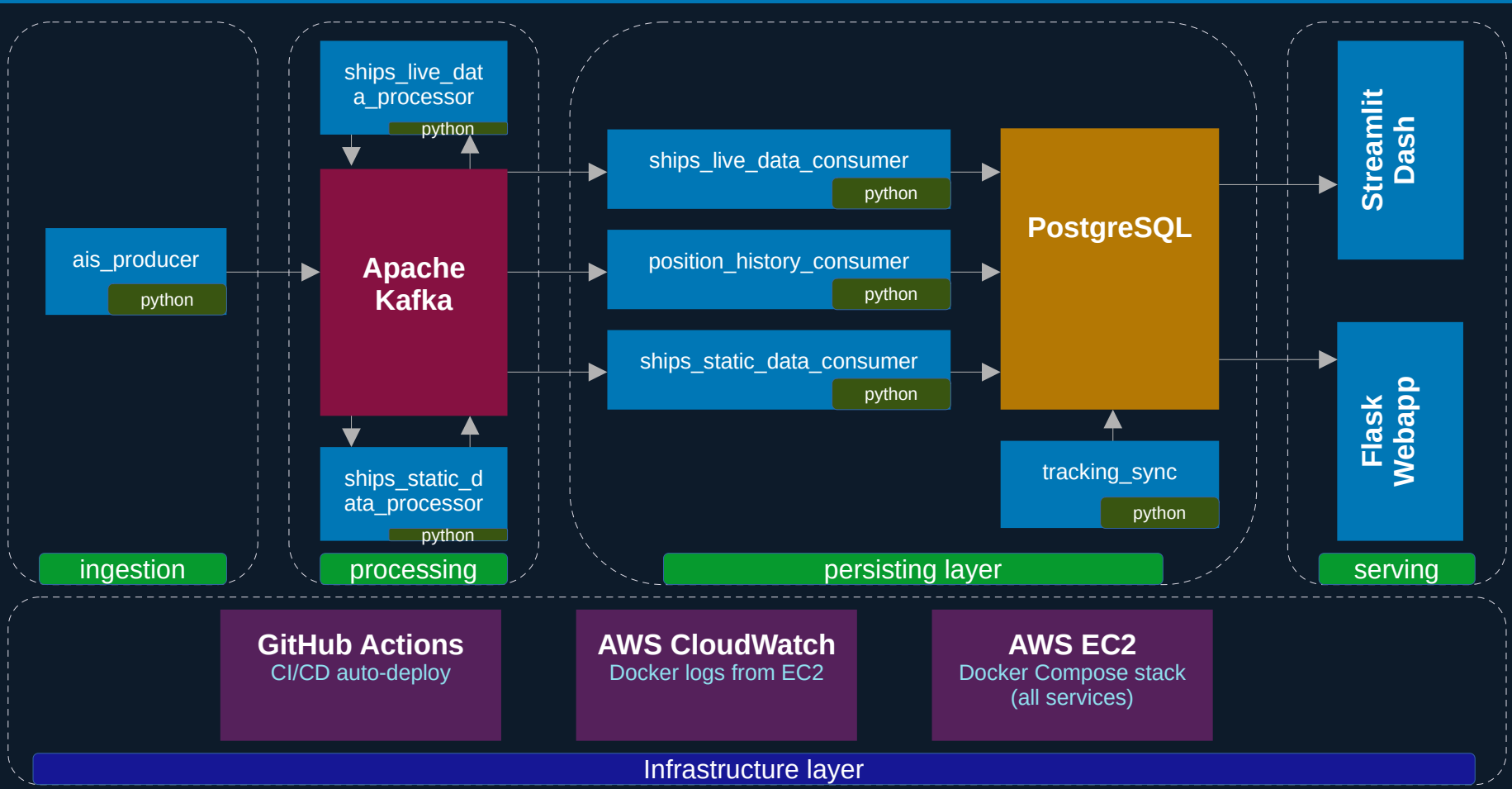
aisstream.io – WebSocket API delivering a real-time global AIS feed (provides shore stations only)

Volume observed

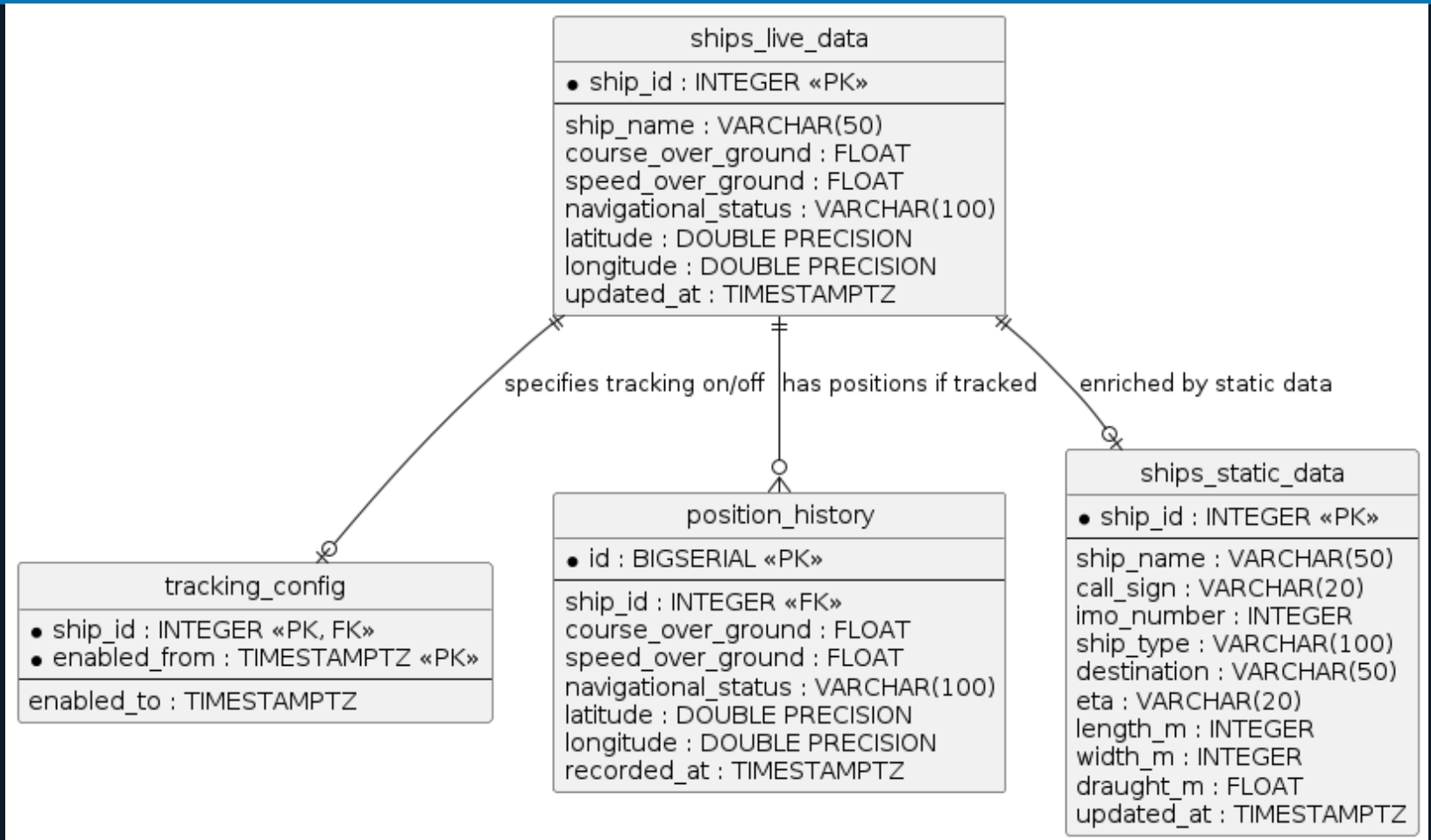
~240–260 msg/s, >3 GBs of data per day


```
{
    "Message": {
        "PositionReport": {
            "Cog": 31.5,
            "CommunicationState": 82117,
            "Latitude": 22.36597,
            "Longitude": 114.106865,
            "MessageID": 1,
            "NavigationalStatus": 15,
            "PositionAccuracy": false,
            "Raim": false,
            "RateOfTurn": 0,
            "RepeatIndicator": 0,
            "Sog": 0,
            "Spare": 0,
            "SpecialManoeuvreIndicator": 0,
            "Timestamp": 44,
            "TrueHeading": 284,
            "UserID": 477995827,
            "Valid": true
        }
    },
    "MessageType": "PositionReport",
    "MetaData": {
        "MMSI": 477995827,
        "MMSI_String": 477995827,
        "ShipName": "GUO SI              ",
        "latitude": 22.36597,
        "longitude": 114.10687,
        "time_utc": "2026-04-24 12:29:45.191749062 +0000 UTC"
    }
}      You, 3 weeks ago • init …
```

System Architecture



Database model · PostgreSQL



Kafka Message Pipeline

ais_producer	► Subscribes to aisstream.io WebSocket · routes each message by MessageType to its own Kafka topic
ships_live_data_processor	► Validates PositionReport · normalises MMSI, lat/lon, speed · publishes to ships_live_data topic
ships_live_data_consumer	► Reads ships_live_data topic · upserts latest position into PostgreSQL ships_live_data table
position_history_consumer	► Reads ships_live_data topic · appends rows for actively-tracked ships · respects tracking_config
ships_static_data_processor	► Parses ShipStaticData · extracts name, type, dimensions · publishes to ships_static_data topic
ships_static_data_consumer	► Reads ships_static_data topic · upserts ship metadata into ships_static_data table
tracking_sync	► Syncs tracked ship IDs from config JSON to PostgreSQL tracking_config on manual trigger

Infrastructure & Deployment

Docker Compose

- Single compose file:
    - ZooKeeper · Kafka · Kafka-UI · PostgreSQL
    - 7 Python apps · Flask webapp · Streamlit dashboard

- Container tracking_sync:
    - Does NOT start with `docker compose up`
    - `option profiles: - tools`

- All other containers: `restart: unless-stopped`

AWS EC2

- Ubuntu instance
    - t3.large
    - region: eu-central-1
- CloudWatch
    - awslogs driver on every container
    - log group ais-ship-tracker
- Security group:
    - SSH inbound
    - 5000 inbound for flask webapp
    - 8501 inbound for streamlit dashboard

GitHub Actions

- provision-ec2.yml
    - manual
    → one-time EC2 setup
- deploy.yml
    - push to main
    → SSH → git pull → docker compose up
- tracking-sync.yml
    - commit to main in path config
    → SSH → git pull → run tracking_sync.py

Secrets & Security

Github secrets:
- provision
    - AWS credentials for AWS CLI
    - KeyPair name
    - SSH deploy key to access GitHub repo from EC2 instance
- deploy/tracking
    - EC2 instance access (user, host)
    - KeyPair key
    - ENV_FILE secret keeps .env out of git
    - SSH deploy key to access GitHub repo from EC2 instance

Tech Stack & Key Learnings

Streaming	Backend	Frontend	Data	DevOps
Apache Kafka 7.5	Python 3.11	Streamlit	PostgreSQL 16	Docker Compose
Confluent ZooKeeper	Flask	MapLibre GL JS	GeoJSON	AWS EC2
kafka-python	psycopg2	Plotly Express	AIS Protocol	GitHub Actions
websockets	asyncio	Pandas	aisstream.io	CloudWatch

Key Learnings

- ► End-to-end streaming: live WebSocket → Kafka → PostgreSQL → interactive dashboards
- ► Multi-consumer architecture enables independent scaling and independent schema evolution
- ► Docker Compose gives full local dev / production parity with a single file
- ► GitHub Actions CI/CD with SSH deploy removes every manual deployment step

⚓ Thank You

AIS Ship Tracker · github.com/thjunge11/aisstream-ship-tracker